BigQuery SQL Interview Practice

25 questions (13 medium, 12 hard). Every question shows the BigQuery solution and its MySQL 8.0+ equivalent for revision.

Each question has: problem, schema, sample input, expected output with ORDER BY, explanation, solution. Expected outputs were produced by running the solution logic on the sample data in SQLite (window functions, CTEs, recursion). Syntax was written for the dialect shown but was not executed on a live MySQL or BigQuery server, so confirm in your own console.

Contents

#	Title	Level	Topic
B1	Order fulfilment status from item statuses	Medium	COUNTIF / conditional aggregation
B2	Top 3 products per category, ties included	Medium	QUALIFY vs subquery, RANK
B3	Weekly retained users	Hard	DATE_TRUNC week, self-join on previous week
B4	Ordered funnel: view -> add_to_cart -> purchase	Hard	First-time per step, conditional counts
B5	First and last event per user (ARRAY_AGG)	Hard	ARRAY_AGG ordered with LIMIT vs window functions
B6	Tag frequency from an array column (UNNEST)	Medium	UNNEST / JSON_TABLE
B7	Year-over-year growth by month	Hard	Self-join on previous year, NULL handling
B8	p50 and p90 delivery time per city	Medium	PERCENTILE_DISC vs ROW_NUMBER nearest-rank
B9	First-touch attribution revenue	Medium	QUALIFY ROW_NUMBER, join
B10	Zero-filled daily series with running total	Hard	GENERATE_DATE_ARRAY vs recursive calendar
B11	Longest gap between consecutive orders	Medium	LAG, DATE_DIFF, QUALIFY
B12	D1 and D7 retention by install date	Hard	Day-offset joins, distinct conditional counts
B13	Top 2 co-purchased products for each product	Hard	Self-join pairs, ranking per product
B14	Pivot quarters into columns	Medium	PIVOT operator vs CASE
B15	Latest non-deleted version per customer	Medium	Dedupe with QUALIFY / ROW_NUMBER
B16	Customers making up the first 80% of revenue (Pareto)	Hard	Running SUM / total, boundary row
B17	Users with 3+ consecutive active weeks	Hard	Gaps and islands on weeks
B18	Price at order time (effective-dated join)	Hard	Range join + latest effective row
B19	Busiest hour per city (ties kept)	Medium	EXTRACT HOUR, RANK
B20	Moving average of a customer's last 3 orders	Medium	ROWS frame
B21	Spend quartiles with NTILE	Medium	NTILE bucket sizes
B22	Total reports under every manager (recursive)	Hard	WITH RECURSIVE subtree count
B23	Signup -> trial -> paid within 7 days (sequence)	Hard	Multi-join event sequence
B24	Leaderboard: ROW_NUMBER vs RANK vs DENSE_RANK	Medium	Ranking functions, PERCENT_RANK
B25	Churn candidates as of a date	Medium	Date boundaries, HAVING

B1 Order fulfilment status from item statuses [Medium]

Each order has several items with status PACKED, SHIPPED, CANCELLED or PENDING. Order status, checked in order: all items CANCELLED -> CANCELLED; every non-cancelled item SHIPPED -> SHIPPED; some SHIPPED -> PARTIAL; some PACKED (none shipped) -> READY; otherwise PENDING. Return order_id, order_status.

Schema

`order_items` (order_id string, item_id int, status string)

Sample input

`order_items`

order_id	item_id	status
O1	1	SHIPPED
O1	2	SHIPPED
O1	3	CANCELLED
O2	4	SHIPPED
O2	5	PACKED
O3	6	PACKED
O3	7	PENDING
O4	8	CANCELLED
O4	9	CANCELLED
O5	10	PENDING
O6	11	SHIPPED

Expected output (ORDER BY order_id)

order_id	order_status
O1	SHIPPED
O2	PARTIAL
O3	READY
O4	CANCELLED
O5	PENDING
O6	SHIPPED

Explanation

- O1: 2 shipped + 1 cancelled -> every live item shipped -> SHIPPED. O2: shipped + packed -> PARTIAL. O3: packed + pending -> READY.
- O4: all cancelled -> CANCELLED. O5: only pending -> PENDING. O6 -> SHIPPED.
- BigQuery: COUNTIF(cond). MySQL: SUM(cond) works because a comparison returns 1 or 0 (or COUNT(CASE WHEN ... THEN 1 END)).

BigQuery

```
select order_id,
       case when countif(status = 'CANCELLED') = count(*)                        then 'CANCELLED'
            when countif(status = 'SHIPPED') = count(*) - countif(status = 'CANCELLED') then 'SHIPPED'
            when countif(status = 'SHIPPED') > 0                                 then 'PARTIAL'
            when countif(status = 'PACKED')  > 0                                 then 'READY'
            else 'PENDING' end as order_status
from order_items
group by order_id
order by order_id;
```

MySQL 8.0+ equivalent

```
select order_id,
       case when sum(status = 'CANCELLED') = count(*)                            then 'CANCELLED'
            when sum(status = 'SHIPPED') = count(*) - sum(status = 'CANCELLED')    then 'SHIPPED'
```

```
            when sum(status = 'SHIPPED') > 0                                then 'PARTIAL'
            when sum(status = 'PACKED')  > 0                                then 'READY'
            else 'PENDING' end as order_status
from order_items
group by order_id
order by order_id;
```

B2 Top 3 products per category, ties included [Medium]

Return the products ranked in the top 3 of their category by units sold. Ties share a rank (RANK, so a rank can be skipped). Return category, product, units, rnk.

Schema

`product_sales` (category string, product string, units int)

Sample input

`product_sales`

category	product	units
Books	A	50
Books	B	40
Books	C	40
Books	D	30
Books	E	10
Toys	X	20
Toys	Y	20
Toys	Z	20
Toys	W	5
Food	F	10

Expected output (ORDER BY category, rnk, product)

category	product	units	rnk
Books	A	50	1
Books	B	40	2
Books	C	40	2
Food	F	10	1
Toys	X	20	1
Toys	Y	20	1
Toys	Z	20	1

Explanation

- Books: A=1, B and C tie at 2, D is rank 4 (RANK skips 3) -> D is excluded. Toys: X, Y, Z tie at 1; W is rank 4.
- BigQuery QUALIFY filters on window functions without a wrapper. MySQL has no QUALIFY: compute the rank in a subquery and filter outside.
- DENSE_RANK would include D (rank 3); pick the function that matches the wording of the question.

BigQuery

```
select category, product, units,
       rank() over (partition by category order by units desc) as rnk
from product_sales
where true                                  -- QUALIFY needs a WHERE/GROUP BY/HAVING in BigQuery
qualify rank() over (partition by category order by units desc) <= 3
order by category, rnk, product;
```

MySQL 8.0+ equivalent

```
select category, product, units, rnk
from (
  select category, product, units,
         rank() over (partition by category order by units desc) as rnk
  from product_sales
) t
where rnk <= 3
order by category, rnk, product;
```

B3 Weekly retained users [Hard]

Weeks start on Monday. For each week return active users, how many of them were also active the previous week, and the percent. Return week_start, active_users, retained, retention_pct.

Schema

`activity` (user_id string, activity_date date)

Sample input

`activity`

user_id	activity_date
u1	2026-10-06
u1	2026-10-08
u1	2026-10-13
u1	2026-10-20
u2	2026-10-07
u2	2026-10-22
u3	2026-10-14
u3	2026-10-21
u4	2026-10-09
u5	2026-10-25

Expected output (ORDER BY week_start)

week_start	active_users	retained	retention_pct
2026-10-05	3	0	0.00
2026-10-12	2	1	50.00
2026-10-19	4	2	50.00

Explanation

- Week of Oct 5: u1, u2, u4 active; nobody in the week before -> 0 retained. Week of Oct 12: u1, u3 active; only u1 was active in Oct 5 -> 50%.
- Week of Oct 19: u1, u2, u3, u5; u1 and u3 were active in Oct 12 -> 50%. u5's Sunday Oct 25 still belongs to the week starting Oct 19.
- COUNT(prev.user_id) counts only matched rows (NULLs from the LEFT JOIN are skipped). MySQL week start: date - WEEKDAY(date) days (Monday = 0).

BigQuery

```
with w as (
  select distinct user_id, date_trunc(activity_date, week(monday)) as wk
  from activity
)
select cur.wk as week_start,
       count(*) as active_users,
       count(prev.user_id) as retained,
       round(100 * safe_divide(count(prev.user_id), count(*)), 2) as retention_pct
from w cur
left join w prev on prev.user_id = cur.user_id
                and prev.wk = date_sub(cur.wk, interval 7 day)
group by cur.wk
order by cur.wk;
```

MySQL 8.0+ equivalent

```
with w as (
  select distinct user_id, date_sub(activity_date, interval weekday(activity_date) day) as wk
  from activity
)
select cur.wk as week_start,
```

```
       count(*) as active_users,
       count(prev.user_id) as retained,
       round(100.0 * count(prev.user_id) / count(*), 2) as retention_pct
from w cur
left join w prev on prev.user_id = cur.user_id
                and prev.wk = date_sub(cur.wk, interval 7 day)
group by cur.wk
order by cur.wk;
```

B4 Ordered funnel: view -> add_to_cart -> purchase [Hard]

A user passes a step only if the step happens after the previous step (using each user's FIRST time for each event). Return step, users, pct_prev (percent of previous step) and pct_top (percent of the first step).

Schema

`events` (user_id string, event_name string, event_time timestamp)

Sample input

`events`

user_id	event_name	event_time
u1	view	2026-10-30 10:00:00
u1	add_to_cart	2026-10-30 10:05:00
u1	purchase	2026-10-30 10:10:00
u2	view	2026-10-30 11:00:00
u2	add_to_cart	2026-10-30 11:03:00
u3	view	2026-10-30 12:00:00
u4	add_to_cart	2026-10-30 09:00:00
u4	view	2026-10-30 10:00:00
u5	view	2026-10-30 13:00:00
u5	add_to_cart	2026-10-30 13:10:00
u5	purchase	2026-10-30 13:20:00
u6	view	2026-10-30 14:00:00
u6	purchase	2026-10-30 14:05:00

Expected output (ORDER BY funnel step)

step	users	pct_prev	pct_top
view	6	NULL	100.00
add_to_cart	3	50.00	50.00
purchase	2	66.67	33.33

Explanation

- 6 users viewed. Add-to-cart after view: u1, u2, u5 (u4 added to cart BEFORE viewing, so fails) -> 3. Purchase after cart after view: u1, u5 -> 2 (u6 skipped the cart).
- Conversions: 3/6 = 50.00% then 2/3 = 66.67% of previous, 33.33% of the top.
- Comparisons with NULL are NULL, so users missing a step drop out of COUNTIF / SUM automatically.

BigQuery

```
with f as (
  select user_id,
         min(if(event_name = 'view',        event_time, null)) as v,
         min(if(event_name = 'add_to_cart', event_time, null)) as c,
         min(if(event_name = 'purchase',    event_time, null)) as p
  from events group by user_id
),
s as (
  select countif(v is not null) as s1, countif(c > v) as s2, countif(p > c and c > v) as s3 from f
)
select step, users, pct_prev, pct_top
from (
  select 1 as n, 'view' as step, s1 as users, cast(null as float64) as pct_prev, 100.0 as pct_top from s
  union all select 2, 'add_to_cart', s2, round(100 * safe_divide(s2, s1), 2),
                                         round(100 * safe_divide(s2, s1), 2) from s
  union all select 3, 'purchase',    s3, round(100 * safe_divide(s3, s2), 2),
                                         round(100 * safe_divide(s3, s1), 2) from s
)
order by n;
```

MySQL 8.0+ equivalent

```
with f as (
  select user_id,
         min(case when event_name = 'view'        then event_time end) as v,
         min(case when event_name = 'add_to_cart' then event_time end) as c,
         min(case when event_name = 'purchase'    then event_time end) as p
  from events group by user_id
),
s as (
  select count(v) as s1,
         coalesce(sum(c > v), 0) as s2,
         coalesce(sum(p > c and c > v), 0) as s3
  from f
)
select step, users, pct_prev, pct_top
from (
  select 1 as n, 'view' as step, s1 as users, null as pct_prev, 100.00 as pct_top from s
  union all select 2, 'add_to_cart', s2, round(100.0 * s2 / s1, 2), round(100.0 * s2 / s1, 2) from s
  union all select 3, 'purchase',    s3, round(100.0 * s3 / s2, 2), round(100.0 * s3 / s1, 2) from s
) t
order by n;
```

Note: Using first occurrence is a simplification. A stricter funnel looks for any cart event after the first view, which needs a join or ARRAY logic.

B5 First and last event per user (ARRAY_AGG) [Hard]

For every user return their first event type, last event type and number of events, but only users whose LAST event is 'purchase'. Return user_id, first_event, last_event, events_count.

Schema

`events` (user_id string, event_time timestamp, event_type string)

Sample input

`events`

user_id	event_time	event_type
u1	2026-10-30 09:00:00	view
u1	2026-10-30 09:05:00	add_to_cart
u1	2026-10-30 09:10:00	purchase
u2	2026-10-30 09:00:00	view
u2	2026-10-30 09:30:00	view
u3	2026-10-30 10:00:00	add_to_cart
u3	2026-10-30 10:20:00	purchase
u4	2026-10-30 11:00:00	purchase
u5	2026-10-30 12:00:00	view
u5	2026-10-30 12:10:00	purchase
u5	2026-10-30 12:20:00	view

Expected output (ORDER BY user_id)

user_id	first_event	last_event	events_count
u1	view	purchase	3
u3	add_to_cart	purchase	2
u4	purchase	purchase	1

Explanation

- u1: view ... purchase, 3 events. u3: add_to_cart ... purchase, 2. u4: single purchase event, so first = last.
- u2 and u5 end on a view and are filtered out. u5 shows why you need the LAST event, not just 'has a purchase'.
- ARRAY_AGG(x ORDER BY t LIMIT 1)[OFFSET(0)] is BigQuery's idiom for first/last value inside GROUP BY. LAST_VALUE needs an explicit full frame, otherwise it returns the current row.

BigQuery

```
select user_id, first_event, last_event, events_count
from (
  select user_id,
         array_agg(event_type order by event_time asc  limit 1)[offset(0)] as first_event,
         array_agg(event_type order by event_time desc limit 1)[offset(0)] as last_event,
         count(*) as events_count
  from events
  group by user_id
)
where last_event = 'purchase'
order by user_id;
```

MySQL 8.0+ equivalent

```
select user_id, first_event, last_event, events_count
from (
  select user_id,
         first_value(event_type) over w as first_event,
         last_value(event_type)  over (partition by user_id order by event_time
                                       rows between unbounded preceding and unbounded following) as last_event,
         count(*) over (partition by user_id) as events_count,
         row_number() over w as rn
```

```
  from events
  window w as (partition by user_id order by event_time)
) t
where rn = 1 and last_event = 'purchase'
order by user_id;
```

B6 Tag frequency from an array column (UNNEST) [Medium]

Each post has a list of tags. Count in how many DISTINCT posts each tag appears (a tag repeated inside one post counts once). Return tag, posts.

Schema

`posts` (post_id int, tags array<string> -- shown as JSON text)

Sample input

`posts`

post_id	tags
1	["sql","python"]
2	["sql"]
3	["python","ml","sql"]
4	["ml"]
5	[]
6	["sql","sql"]

Expected output (ORDER BY posts DESC, tag)

tag	posts
sql	4
ml	2
python	2

Explanation

- Post 6 has 'sql' twice; DISTINCT post_id makes it count once. Post 5 has an empty array and disappears from the UNNEST cross join.
- sql is in posts 1, 2, 3, 6 -> 4. python in 1, 3 -> 2. ml in 3, 4 -> 2 (tie broken alphabetically).
- UNNEST turns an array into rows (implicit cross join with the parent row). JSON_TABLE is MySQL's closest match for JSON arrays.

BigQuery

```
select tag, count(distinct post_id) as posts
from posts, unnest(tags) as tag
group by tag
order by posts desc, tag;
```

MySQL 8.0+ equivalent

```
select j.tag, count(distinct p.post_id) as posts
from posts p,
     json_table(p.tags, '$[*]' columns (tag varchar(30) path '$')) j
group by j.tag
order by posts desc, j.tag;
```

Note: The MySQL version needs a live MySQL 8 server; I verified the expected output with the equivalent SQLite json_each query.

B7 Year-over-year growth by month [Hard]

For every month of 2026 return revenue, revenue in the same month of 2025 (NULL if none) and YoY growth percent (NULL if no prior year data). Return month, revenue, same_month_last_year, yoy_pct.

Schema

`orders` (order_date date, amount int)

Sample input

`orders`

order_date	amount
2025-01-10	100
2025-02-11	200
2025-03-12	150
2026-01-08	100
2026-01-22	50
2026-02-15	180
2026-04-04	300

Expected output (ORDER BY month)

month	revenue	same_month_last_year	yoy_pct
2026-01	150	100	50.00
2026-02	180	200	-10.00
2026-04	300	NULL	NULL

Explanation

- Jan: 150 vs 100 -> +50.00%. Feb: 180 vs 200 -> -10.00%. Apr: 300 vs nothing in Apr 2025 -> NULL, not 0 or infinite growth.
- March 2026 has no sales, so there is no March row at all (add a calendar spine if you need one).
- LAG(rev, 12) would be wrong here because months with no sales are missing rows; joining on (year - 1, month) is safe.

BigQuery

```
with m as (
  select format_date('%Y-%m', order_date) as ym,
         extract(year from order_date)  as y,
         extract(month from order_date) as mo,
         sum(amount) as rev
  from orders
  group by 1, 2, 3
)
select cur.ym as month, cur.rev as revenue, prev.rev as same_month_last_year,
       round(100 * safe_divide(cur.rev - prev.rev, prev.rev), 2) as yoy_pct
from m cur
left join m prev on prev.y = cur.y - 1 and prev.mo = cur.mo
where cur.y = 2026
order by cur.ym;
```

MySQL 8.0+ equivalent

```
with m as (
  select date_format(order_date, '%Y-%m') as ym,
         year(order_date)  as y,
         month(order_date) as mo,
         sum(amount) as rev
  from orders
  group by date_format(order_date, '%Y-%m'), year(order_date), month(order_date)
)
select cur.ym as month, cur.rev as revenue, prev.rev as same_month_last_year,
       round(100.0 * (cur.rev - prev.rev) / prev.rev, 2) as yoy_pct
from m cur
left join m prev on prev.y = cur.y - 1 and prev.mo = cur.mo
```

```
where cur.y = 2026
order by cur.ym;
```

B8 p50 and p90 delivery time per city [Medium]

Return the nearest-rank percentiles (smallest value whose cumulative share is >= p) of delivery minutes per city: p50 and p90. Return city, p50, p90.

Schema

`deliveries` (delivery_id int, city string, minutes int)

Sample input

`deliveries`

delivery_id	city	minutes
1	Delhi	20
2	Delhi	25
3	Delhi	30
4	Delhi	35
5	Delhi	40
6	Delhi	45
7	Delhi	50
8	Delhi	55
9	Delhi	60
10	Delhi	120
11	Mumbai	15
12	Mumbai	20
13	Mumbai	25
14	Mumbai	30
15	Mumbai	90
16	Pune	33

Expected output (ORDER BY city)

city	p50	p90
Delhi	40	60
Mumbai	25	90
Pune	33	33

Explanation

- Nearest rank: position = ceil(p * n). Delhi n = 10: p50 = 5th = 40, p90 = 9th = 60 (the 120 outlier is ignored by p90). Mumbai n = 5: p50 = ceil(2.5) = 3rd = 25, p90 = 5th = 90. Pune: 33.
- BigQuery PERCENTILE_DISC is the analytic version of the same definition. PERCENTILE_CONT would interpolate and give different numbers (e.g. 42.5 for Delhi p50).
- MySQL has no percentile function; rank rows and pick by position.

BigQuery

```
select distinct city,
       percentile_disc(minutes, 0.5) over (partition by city) as p50,
       percentile_disc(minutes, 0.9) over (partition by city) as p90
from deliveries
order by city;
```

MySQL 8.0+ equivalent

```
with r as (
  select city, minutes,
         row_number() over (partition by city order by minutes) as rn,
         count(*)     over (partition by city) as cnt
  from deliveries
```

```
)
select city,
       max(case when rn = ceil(cnt * 0.5) then minutes end) as p50,
       max(case when rn = ceil(cnt * 0.9) then minutes end) as p90
from r
group by city
order by city;
```

B9 First-touch attribution revenue [Medium]

Attribute each customer's total order revenue to the channel of their FIRST touchpoint. Return channel, customers (with orders) and revenue.

Schema

`touchpoints` (customer_id string, touch_time date, channel string)
`orders` (order_id int, customer_id string, amount int)

Sample input

`touchpoints`

customer_id	touch_time	channel
c1	2026-09-01	Email
c1	2026-09-05	Ads
c2	2026-09-02	Ads
c2	2026-09-03	Email
c3	2026-09-04	Organic
c4	2026-09-06	Ads

`orders`

order_id	customer_id	amount
1	c1	100
2	c1	50
3	c2	200
4	c3	80

Expected output (ORDER BY revenue DESC)

channel	customers	revenue
Ads	1	200
Email	1	150
Organic	1	80

Explanation

- c1 first touched Email, c2 first touched Ads, c3 Organic; c4 has no orders and drops out in the inner join.
- Revenue: Ads (c2) 200, Email (c1) 150, Organic (c3) 80.
- Aggregate orders to one row per customer BEFORE joining, so touchpoints do not multiply order revenue.

BigQuery

```
with ft as (
  select customer_id, channel
  from touchpoints
  where true
  qualify row_number() over (partition by customer_id order by touch_time) = 1
),
o as (select customer_id, sum(amount) as rev from orders group by customer_id)
select ft.channel, count(*) as customers, sum(o.rev) as revenue
from ft join o using (customer_id)
group by ft.channel
order by revenue desc;
```

MySQL 8.0+ equivalent

```
with ft as (
  select customer_id, channel
  from (select customer_id, channel,
               row_number() over (partition by customer_id order by touch_time) as rn
        from touchpoints) t
  where rn = 1
),
```

```
o as (select customer_id, sum(amount) as rev from orders group by customer_id)
select ft.channel, count(*) as customers, sum(o.rev) as revenue
from ft join o on o.customer_id = ft.customer_id
group by ft.channel
order by revenue desc;
```

B10 Zero-filled daily series with running total [Hard]

Return one row for every date from 2026-10-01 to 2026-10-07, with that day's revenue (0 when no sales) and the cumulative revenue. Return sale_date, revenue, cumulative_revenue.

Schema

`sales` (sale_date date, amount int)

Sample input

`sales`

sale_date	amount
2026-10-01	50
2026-10-01	25
2026-10-03	40
2026-10-04	10
2026-10-07	100

Expected output (ORDER BY sale_date)

sale_date	revenue	cumulative_revenue
2026-10-01	75	75
2026-10-02	0	75
2026-10-03	40	115
2026-10-04	10	125
2026-10-05	0	125
2026-10-06	0	125
2026-10-07	100	225

Explanation

- Oct 1 has two sales (75). Oct 2, 5 and 6 have no rows in `sales` but still appear with 0.
- A window function can wrap an aggregate: SUM(SUM(...)) OVER (ORDER BY d) runs after GROUP BY.
- BigQuery builds the calendar with GENERATE_DATE_ARRAY + UNNEST; MySQL uses a recursive CTE.

BigQuery

```
select d as sale_date,
       coalesce(sum(s.amount), 0) as revenue,
       sum(coalesce(sum(s.amount), 0)) over (order by d) as cumulative_revenue
from unnest(generate_date_array(date '2026-10-01', date '2026-10-07')) as d
left join sales s on s.sale_date = d
group by d
order by d;
```

MySQL 8.0+ equivalent

```
with recursive cal as (
  select date('2026-10-01') as d
  union all
  select date_add(d, interval 1 day) from cal where d < date('2026-10-07')
)
select c.d as sale_date,
       coalesce(sum(s.amount), 0) as revenue,
       sum(coalesce(sum(s.amount), 0)) over (order by c.d) as cumulative_revenue
from cal c
left join sales s on s.sale_date = c.d
group by c.d
order by c.d;
```

B11 Longest gap between consecutive orders [Medium]

For each customer with at least 2 orders return the longest gap in days between two consecutive orders and the two dates. Ties: earliest gap. Return customer_id, max_gap_days, gap_start, gap_end.

Schema

`orders` (customer_id string, order_date date)

Sample input

`orders`

customer_id	order_date
c1	2026-01-01
c1	2026-01-10
c1	2026-01-12
c1	2026-02-20
c2	2026-03-01
c2	2026-03-05
c3	2026-04-01
c4	2026-05-01
c4	2026-05-08
c4	2026-05-15

Expected output (ORDER BY customer_id)

customer_id	max_gap_days	gap_start	gap_end
c1	39	2026-01-12	2026-02-20
c2	4	2026-03-01	2026-03-05
c4	7	2026-05-01	2026-05-08

Explanation

- c1 gaps are 9, 2, 39 -> 39 days (Jan 12 to Feb 20). c3 has a single order, no gap, so it is excluded.
- c4 gaps are 7 and 7: tie broken by the earlier start -> May 1 to May 8.
- Argument order: BigQuery DATE_DIFF(later, earlier, DAY); MySQL DATEDIFF(later, earlier).

BigQuery

```
with g as (
  select customer_id,
         lag(order_date) over w as gap_start,
         order_date as gap_end,
         date_diff(order_date, lag(order_date) over w, day) as gap_days
  from orders
  window w as (partition by customer_id order by order_date)
)
select customer_id, gap_days as max_gap_days, gap_start, gap_end
from g
where gap_days is not null
qualify row_number() over (partition by customer_id order by gap_days desc, gap_start) = 1
order by customer_id;
```

MySQL 8.0+ equivalent

```
with g as (
  select customer_id,
         lag(order_date) over w as gap_start,
         order_date as gap_end,
         datediff(order_date, lag(order_date) over w) as gap_days
  from orders
  window w as (partition by customer_id order by order_date)
),
r as (
  select *, row_number() over (partition by customer_id order by gap_days desc, gap_start) as rn
```

```
  from g where gap_days is not null
)
select customer_id, gap_days as max_gap_days, gap_start, gap_end
from r where rn = 1
order by customer_id;
```

B12 D1 and D7 retention by install date [Hard]

D1 retention = share of installs that had a session exactly 1 day after install; D7 = exactly 7 days after. Return install_date, installs, d1_pct, d7_pct (2 decimals).

Schema

`installs` (user_id string, install_date date)
`sessions` (user_id string, session_date date)

Sample input

`installs`

user_id	install_date
u1	2026-10-01
u2	2026-10-01
u3	2026-10-01
u4	2026-10-01
u5	2026-10-02
u6	2026-10-02

`sessions`

user_id	session_date
u1	2026-10-02
u1	2026-10-08
u2	2026-10-02
u2	2026-10-08
u3	2026-10-08
u5	2026-10-03
u6	2026-10-09

Expected output (ORDER BY install_date)

install_date	installs	d1_pct	d7_pct
2026-10-01	4	50.00	75.00
2026-10-02	2	50.00	50.00

Explanation

- Oct 1 cohort (4 installs): D1 = u1, u2 (50.00). D7 = Oct 8 sessions of u1, u2, u3 (75.00). u4 never returned.
- Oct 2 cohort (2 installs): D1 = u5 (50.00). D7 = u6 on Oct 9 (50.00).
- COUNT(DISTINCT user) is essential: the join repeats each install once per session row.

BigQuery

```
select i.install_date,
       count(distinct i.user_id) as installs,
       round(100 * safe_divide(
         count(distinct if(s.session_date = date_add(i.install_date, interval 1 day), i.user_id, null)),
         count(distinct i.user_id)), 2) as d1_pct,
       round(100 * safe_divide(
         count(distinct if(s.session_date = date_add(i.install_date, interval 7 day), i.user_id, null)),
         count(distinct i.user_id)), 2) as d7_pct
from installs i
left join sessions s on s.user_id = i.user_id
group by i.install_date
order by i.install_date;
```

MySQL 8.0+ equivalent

```
select i.install_date,
       count(distinct i.user_id) as installs,
       round(100.0 * count(distinct case when s.session_date = date_add(i.install_date, interval 1 day)
```

```
                                        then i.user_id end) / count(distinct i.user_id), 2) as d1_pct,
       round(100.0 * count(distinct case when s.session_date = date_add(i.install_date, interval 7 day)
                                        then i.user_id end) / count(distinct i.user_id), 2) as d7_pct
from installs i
left join sessions s on s.user_id = i.user_id
group by i.install_date
order by i.install_date;
```

B13 Top 2 co-purchased products for each product [Hard]

For every product return the 2 products most often bought in the same order as it (ties by product name). Return product, bought_with, orders_together, rn.

Schema

`order_items` (order_id string, product string)

Sample input

`order_items`

order_id	product
O1	A
O1	B
O1	C
O2	A
O2	B
O3	A
O3	C
O4	B
O4	C
O4	D
O5	A
O5	D
O6	D

Expected output (ORDER BY product, rn)

product	bought_with	orders_together	rn
A	B	2	1
A	C	2	2
B	A	2	1
B	C	2	2
C	A	2	1
C	B	2	2
D	A	1	1
D	B	1	2

Explanation

- Pair counts: A-B 2, A-C 2, A-D 1, B-C 2, B-D 1, C-D 1. Unlike Q4, pairs are directional here (A->B and B->A both exist) so each product gets its own list.
- D: its three partners all tie at 1, so name order picks A and B.
- Use `!=` / `<>` (not `<`) to keep both directions; use `<` when each unordered pair should appear once.

BigQuery

```
with p as (
  select a.product as product, b.product as bought_with, count(*) as orders_together
  from order_items a
  join order_items b on a.order_id = b.order_id and a.product != b.product
  group by 1, 2
)
select product, bought_with, orders_together,
       row_number() over (partition by product order by orders_together desc, bought_with) as rn
from p
where true
qualify row_number() over (partition by product order by orders_together desc, bought_with) <= 2
order by product, rn;
```

MySQL 8.0+ equivalent

```
with p as (
  select a.product as product, b.product as bought_with, count(*) as orders_together
  from order_items a
  join order_items b on a.order_id = b.order_id and a.product <> b.product
  group by a.product, b.product
),
r as (
  select *, row_number() over (partition by product order by orders_together desc, bought_with) as rn from p
)
select product, bought_with, orders_together, rn
from r where rn <= 2
order by product, rn;
```

B14 Pivot quarters into columns [Medium]

Return one row per region with columns q1..q4 containing total sales (0 if missing). Return region, q1, q2, q3, q4.

Schema

`sales` (region string, qtr string, amount int)

Sample input

`sales`

region	qtr	amount
North	Q1	100
North	Q1	20
North	Q2	150
North	Q3	90
South	Q1	50
South	Q2	60
South	Q3	70
South	Q4	80

Expected output (ORDER BY region)

region	q1	q2	q3	q4
North	120	150	90	0
South	50	60	70	80

Explanation

- North Q1 = 100 + 20 = 120; North has no Q4 row -> 0. South has all four quarters.
- BigQuery PIVOT lists the column values explicitly and returns NULL for missing cells, hence IFNULL. MySQL uses one SUM(CASE) per column.
- Select only the columns the pivot needs in the inner query; any extra column becomes an extra grouping key.

BigQuery

```
select region, ifnull(Q1, 0) as q1, ifnull(Q2, 0) as q2, ifnull(Q3, 0) as q3, ifnull(Q4, 0) as q4
from (select region, qtr, amount from sales)
pivot (sum(amount) for qtr in ('Q1', 'Q2', 'Q3', 'Q4'))
order by region;
```

MySQL 8.0+ equivalent

```
select region,
       sum(case when qtr = 'Q1' then amount else 0 end) as q1,
       sum(case when qtr = 'Q2' then amount else 0 end) as q2,
       sum(case when qtr = 'Q3' then amount else 0 end) as q3,
       sum(case when qtr = 'Q4' then amount else 0 end) as q4
from sales
group by region
order by region;
```

B15 Latest non-deleted version per customer [Medium]

A change log keeps every version of a customer. Return the latest version per customer, but drop customers whose latest version is marked deleted. Return customer_id, email, updated_at.

Schema

`customer_updates` (customer_id int, email string, updated_at date, is_deleted bool)

Sample input

`customer_updates`

customer_id	email	updated_at	is_deleted
1	a@x.com	2026-01-01	0
1	a2@x.com	2026-02-01	0
2	b@x.com	2026-01-05	0
2	b@x.com	2026-02-05	1
3	c@x.com	2026-01-10	0
4	d@x.com	2026-01-01	0
4	d2@x.com	2026-03-01	0

Expected output (ORDER BY customer_id)

customer_id	email	updated_at
1	a2@x.com	2026-02-01
3	c@x.com	2026-01-10
4	d2@x.com	2026-03-01

Explanation

- Customer 1 -> a2@x.com (Feb version). Customer 2's latest row is deleted, so customer 2 disappears entirely.
- Order of operations matters: pick the latest row first, THEN drop deleted. Filtering deleted rows first would resurrect customer 2's older version.
- Add a tiebreaker column to ORDER BY if two versions can share a timestamp.

BigQuery

```
select customer_id, email, updated_at
from (
  select customer_id, email, updated_at, is_deleted
  from customer_updates
  where true
  qualify row_number() over (partition by customer_id order by updated_at desc) = 1
)
where not is_deleted
order by customer_id;
```

MySQL 8.0+ equivalent

```
select customer_id, email, updated_at
from (
  select customer_id, email, updated_at, is_deleted,
         row_number() over (partition by customer_id order by updated_at desc) as rn
  from customer_updates
) t
where rn = 1 and is_deleted = 0
order by customer_id;
```

B16 Customers making up the first 80% of revenue (Pareto) [Hard]

Rank customers by revenue (tie: customer_id). Return the smallest set of top customers whose cumulative revenue reaches at least 80% of the total, including the customer that crosses the line. Return customer_id, revenue, cum_pct.

Schema

`customers` (customer_id string, revenue int)

Sample input

`customers`

customer_id	revenue
c1	400
c2	250
c3	150
c4	100
c5	60
c6	40

Expected output (ORDER BY revenue DESC, customer_id)

customer_id	revenue	cum_pct
c1	400	40.00
c2	250	65.00
c3	150	80.00

Explanation

- Total 1000. Cumulative: 400, 650, 800, 900, 960, 1000. c3 lands exactly on 80% and is the last one needed.
- A row is included when the cumulative revenue BEFORE it is below 80%. c4 starts at 800 (80%), not below, so it is excluded.
- 5 * (cum - revenue) < 4 * total is the same as `< 0.8 * total` without floating-point risk.

BigQuery

```
with c as (
  select customer_id, revenue,
         sum(revenue) over (order by revenue desc, customer_id) as cum,
         sum(revenue) over () as total
  from customers
)
select customer_id, revenue, round(100 * cum / total, 2) as cum_pct
from c
where 5 * (cum - revenue) < 4 * total          -- cumulative BEFORE this row is still under 80%
order by revenue desc, customer_id;
```

MySQL 8.0+ equivalent

```
with c as (
  select customer_id, revenue,
         sum(revenue) over (order by revenue desc, customer_id) as cum,
         sum(revenue) over () as total
  from customers
)
select customer_id, revenue, round(100.0 * cum / total, 2) as cum_pct
from c
where 5 * (cum - revenue) < 4 * total
order by revenue desc, customer_id;
```

B17 Users with 3+ consecutive active weeks [Hard]

Find users who were active in at least 3 consecutive weeks (weeks start Monday) and return their longest consecutive-week streak. Return user_id, longest_week_streak.

Schema

`activity` (user_id string, activity_date date)

Sample input

`activity`

user_id	activity_date
u1	2026-10-06
u1	2026-10-14
u1	2026-10-21
u1	2026-10-28
u2	2026-10-06
u2	2026-10-14
u2	2026-10-28
u3	2026-10-13
u3	2026-10-22
u3	2026-10-27
u4	2026-10-08

Expected output (ORDER BY longest_week_streak DESC, user_id)

user_id	longest_week_streak
u1	4
u3	3

Explanation

- u1 is active in 4 consecutive weeks (Oct 5, 12, 19, 26). u3 is active in weeks Oct 12, 19, 26 -> 3. u2 skips a week (2 + 1), so max is 2 -> excluded.
- Same gaps-and-islands trick as the daily streak, but the unit is a week, so subtract rn * 7 days from the week start.
- Truncate dates to weeks first and DISTINCT them, otherwise several visits in one week break the row numbers.

BigQuery

```
with w as (
  select distinct user_id, date_trunc(activity_date, week(monday)) as wk from activity
),
r as (
  select user_id, wk, row_number() over (partition by user_id order by wk) as rn from w
),
g as (
  select user_id, count(*) as len
  from r
  group by user_id, date_sub(wk, interval rn * 7 day)      -- constant inside a streak
)
select user_id, max(len) as longest_week_streak
from g
group by user_id
having max(len) >= 3
order by longest_week_streak desc, user_id;
```

MySQL 8.0+ equivalent

```
with w as (
  select distinct user_id, date_sub(activity_date, interval weekday(activity_date) day) as wk from activity
),
r as (
  select user_id, wk, row_number() over (partition by user_id order by wk) as rn from w
```

```
),
g as (
  select user_id, count(*) as len
  from r
  group by user_id, date_sub(wk, interval rn * 7 day)
)
select user_id, max(len) as longest_week_streak
from g
group by user_id
having max(len) >= 3
order by longest_week_streak desc, user_id;
```

B18 Price at order time (effective-dated join) [Hard]

Prices change over time. For each order use the latest price whose effective_from is on or before the order date. Orders with no applicable price keep NULL. Return order_id, product_id, price_used, revenue (price x qty).

Schema

`prices` (product_id string, price int, effective_from date)
`orders` (order_id int, product_id string, order_date date, qty int)

Sample input

`prices`

product_id	price	effective_from
P1	10	2026-01-01
P1	12	2026-03-01
P1	15	2026-06-01
P2	100	2026-02-01

`orders`

order_id	product_id	order_date	qty
1	P1	2026-02-15	2
2	P1	2026-03-01	1
3	P1	2026-07-01	3
4	P2	2026-01-15	1
5	P2	2026-02-01	1

Expected output (ORDER BY order_id)

order_id	product_id	price_used	revenue
1	P1	10	20
2	P1	12	12
3	P1	15	45
4	P2	NULL	NULL
5	P2	100	100

Explanation

- Join every order to ALL prices that started on or before the order date, then keep the most recent one per order.
- Order 2 (Mar 1) uses 12: the boundary date counts (<=). Order 3 uses 15. Order 4 is before P2's first price -> NULL price and revenue.
- LEFT JOIN keeps order 4; ROW_NUMBER still returns its single NULL row as rn = 1.

BigQuery

```
select order_id, product_id, price as price_used, price * qty as revenue
from (
  select o.order_id, o.product_id, o.qty, p.price
  from orders o
  left join prices p on p.product_id = o.product_id and p.effective_from <= o.order_date
  where true
  qualify row_number() over (partition by o.order_id order by p.effective_from desc) = 1
)
order by order_id;
```

MySQL 8.0+ equivalent

```
select order_id, product_id, price as price_used, price * qty as revenue
from (
  select o.order_id, o.product_id, o.qty, p.price,
         row_number() over (partition by o.order_id order by p.effective_from desc) as rn
  from orders o
  left join prices p on p.product_id = o.product_id and p.effective_from <= o.order_date
```

```
) t
where rn = 1
order by order_id;
```

B19 Busiest hour per city (ties kept) [Medium]

For each city return the hour(s) of day with the most trips. If several hours tie, return all of them. Return city, busiest_hour, trips.

Schema

`trips` (trip_id int, city string, pickup_time timestamp)

Sample input

`trips`

trip_id	city	pickup_time
1	Delhi	2026-10-30 08:05:00
2	Delhi	2026-10-30 08:20:00
3	Delhi	2026-10-30 08:50:00
4	Delhi	2026-10-30 18:10:00
5	Delhi	2026-10-30 18:15:00
6	Delhi	2026-10-30 18:55:00
7	Delhi	2026-10-30 12:00:00
8	Mumbai	2026-10-30 09:01:00
9	Mumbai	2026-10-30 09:30:00
10	Mumbai	2026-10-30 20:15:00
11	Pune	2026-10-30 07:45:00

Expected output (ORDER BY city, busiest_hour)

city	busiest_hour	trips
Delhi	8	3
Delhi	18	3
Mumbai	9	2
Pune	7	1

Explanation

- Delhi: hour 8 and hour 18 both have 3 trips (hour 12 has 1) -> both returned because RANK keeps ties. Mumbai: hour 9 (2). Pune: hour 7 (1).
- BigQuery EXTRACT(HOUR FROM ts) uses UTC for TIMESTAMP unless you pass a time zone; MySQL HOUR(ts).
- Aggregate to (city, hour) first, then rank those counts.

BigQuery

```
with h as (
  select city, extract(hour from pickup_time) as hr, count(*) as trips
  from trips
  group by city, hr
)
select city, hr as busiest_hour, trips
from h
where true
qualify rank() over (partition by city order by trips desc) = 1
order by city, busiest_hour;
```

MySQL 8.0+ equivalent

```
with h as (
  select city, hour(pickup_time) as hr, count(*) as trips
  from trips
  group by city, hour(pickup_time)
),
r as (select *, rank() over (partition by city order by trips desc) as rk from h)
select city, hr as busiest_hour, trips
from r where rk = 1
```

```
order by city, busiest_hour;
```

B20 Moving average of a customer's last 3 orders [Medium]

For each order return its sequence number for the customer and the average amount of that order and the two previous orders of the same customer. Return customer_id, order_id, amount, order_no, moving_avg_3.

Schema

`orders` (customer_id string, order_id int, order_date date, amount int)

Sample input

`orders`

customer_id	order_id	order_date	amount
c1	101	2026-01-01	100
c1	102	2026-01-05	200
c1	103	2026-01-09	300
c1	104	2026-01-15	400
c2	201	2026-02-01	50
c2	202	2026-02-03	150

Expected output (ORDER BY customer_id, order_no)

customer_id	order_id	amount	order_no	moving_avg_3
c1	101	100	1	100.00
c1	102	200	2	150.00
c1	103	300	3	200.00
c1	104	400	4	300.00
c2	201	50	1	50.00
c2	202	150	2	100.00

Explanation

- c1 order 3: (100+200+300)/3 = 200; order 4: (200+300+400)/3 = 300. The first two orders average over what exists (100, then 150).
- A named window can be extended with a frame: `over (w rows between 2 preceding and current row)`. Same syntax in both engines.
- ROWS counts rows, which is what 'last 3 orders' means; use RANGE with dates for 'last 3 days'.

BigQuery

```
select customer_id, order_id, amount,
       row_number() over w as order_no,
       round(avg(amount) over (w rows between 2 preceding and current row), 2) as moving_avg_3
from orders
window w as (partition by customer_id order by order_date, order_id)
order by customer_id, order_no;
```

MySQL 8.0+ equivalent

```
select customer_id, order_id, amount,
       row_number() over w as order_no,
       round(avg(amount) over (w rows between 2 preceding and current row), 2) as moving_avg_3
from orders
window w as (partition by customer_id order by order_date, order_id)
order by customer_id, order_no;
```

B21 Spend quartiles with NTILE [Medium]

Split customers into 4 spend buckets with NTILE (1 = lowest spenders). Return quartile, customers, min_spend, max_spend, avg_spend (2 decimals).

Schema

`customers` (customer_id int, total_spend int)

Sample input

`customers`

customer_id	total_spend
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90

Expected output (ORDER BY quartile)

quartile	customers	min_spend	max_spend	avg_spend
1	3	10	30	20.00
2	2	40	50	45.00
3	2	60	70	65.00
4	2	80	90	85.00

Explanation

- 9 customers do not divide evenly by 4: NTILE gives the extra row to the earliest buckets, so sizes are 3, 2, 2, 2.
- Bucket 1 = 10, 20, 30 (avg 20.00). Bucket 4 = 80, 90 (avg 85.00).
- NTILE splits by row count, not by value ranges. Ties can straddle buckets, so add a tiebreaker to ORDER BY for stable results.

BigQuery

```
with q as (
  select customer_id, total_spend, ntile(4) over (order by total_spend) as quartile
  from customers
)
select quartile, count(*) as customers, min(total_spend) as min_spend,
       max(total_spend) as max_spend, round(avg(total_spend), 2) as avg_spend
from q
group by quartile
order by quartile;
```

MySQL 8.0+ equivalent

```
with q as (
  select customer_id, total_spend, ntile(4) over (order by total_spend) as quartile
  from customers
)
select quartile, count(*) as customers, min(total_spend) as min_spend,
       max(total_spend) as max_spend, round(avg(total_spend), 2) as avg_spend
from q
group by quartile
order by quartile;
```

B22 Total reports under every manager (recursive) [Hard]

For every employee who manages anyone, return the number of direct and indirect reports. Return emp_name, total_reports.

Schema

`employees` (emp_id int, emp_name string, manager_id int)

Sample input

`employees`

emp_id	emp_name	manager_id
1	Asha	NULL
2	Ben	1
3	Cara	1
4	Dev	2
5	Esha	2
6	Farid	4
7	Gita	3

Expected output (ORDER BY total_reports DESC, emp_name)

emp_name	total_reports
Asha	6
Ben	3
Cara	1
Dev	1

Explanation

- Start every employee as the root of their own subtree and expand downwards; each (root, descendant) pair is one row.
- Asha: Ben, Cara, Dev, Esha, Farid, Gita = 6. Ben: Dev, Esha, Farid = 3. Cara: Gita = 1. Dev: Farid = 1. Leaves are excluded by HAVING.
- `count(*) - 1` removes the root's own row. Both BigQuery and MySQL require the RECURSIVE keyword.

BigQuery

```
with recursive t as (
  select emp_id as root, emp_id as cur from employees           -- everyone is their own root
  union all
  select t.root, e.emp_id
  from employees e join t on e.manager_id = t.cur               -- walk downwards
)
select e.emp_name, count(*) - 1 as total_reports
from t join employees e on e.emp_id = t.root
group by e.emp_id, e.emp_name
having count(*) > 1
order by total_reports desc, e.emp_name;
```

MySQL 8.0+ equivalent

```
with recursive t as (
  select emp_id as root, emp_id as cur from employees
  union all
  select t.root, e.emp_id
  from employees e join t on e.manager_id = t.cur
)
select e.emp_name, count(*) - 1 as total_reports
from t join employees e on e.emp_id = t.root
group by e.emp_id, e.emp_name
having count(*) > 1
order by total_reports desc, e.emp_name;
```

B23 Signup -> trial -> paid within 7 days (sequence) [Hard]

Find users who signed up, then started a trial on a LATER date, then paid on a LATER date than the trial, with payment at most 7 days after signup. Return user_id and days_to_paid.

Schema

`events` (user_id string, event_name string, event_date date)

Sample input

`events`

user_id	event_name	event_date
u1	signup	2026-10-01
u1	trial	2026-10-03
u1	paid	2026-10-06
u2	signup	2026-10-01
u2	trial	2026-10-02
u2	paid	2026-10-12
u3	trial	2026-10-01
u3	signup	2026-10-02
u3	paid	2026-10-03
u4	signup	2026-10-05
u4	trial	2026-10-05
u4	paid	2026-10-07
u5	signup	2026-10-01
u5	trial	2026-10-04
u5	paid	2026-10-08

Expected output (ORDER BY user_id)

user_id	days_to_paid
u1	5
u5	7

Explanation

- u1: 5 days, in order -> yes. u5: exactly 7 days -> yes (<= 7). u2 paid after 11 days -> no.
- u3 did trial before signup -> wrong order. u4 started the trial on the SAME day as signup, which is not strictly later -> no.
- Each step is a self-join on the same table with a time condition; GROUP BY + MIN protects against users with repeated events.

BigQuery

```
select s.user_id, min(date_diff(p.event_date, s.event_date, day)) as days_to_paid
from events s
join events t on t.user_id = s.user_id and t.event_name = 'trial' and t.event_date > s.event_date
join events p on p.user_id = s.user_id and p.event_name = 'paid'  and p.event_date > t.event_date
where s.event_name = 'signup'
  and date_diff(p.event_date, s.event_date, day) <= 7
group by s.user_id
order by s.user_id;
```

MySQL 8.0+ equivalent

```
select s.user_id, min(datediff(p.event_date, s.event_date)) as days_to_paid
from events s
join events t on t.user_id = s.user_id and t.event_name = 'trial' and t.event_date > s.event_date
join events p on p.user_id = s.user_id and p.event_name = 'paid'  and p.event_date > t.event_date
where s.event_name = 'signup'
  and datediff(p.event_date, s.event_date) <= 7
group by s.user_id
```

```
order by s.user_id;
```

B24 Leaderboard: ROW_NUMBER vs RANK vs DENSE_RANK [Medium]

Show each player's ROW_NUMBER (tiebreak by name), RANK, DENSE_RANK by score (highest first) and the percentile 'beats x% of players' using PERCENT_RANK on ascending score. Return player, score, rn, rnk, drnk, pct_beaten.

Schema

`scores` (player string, score int)

Sample input

`scores`

player	score
A	90
B	85
C	85
D	70
E	70
F	60

Expected output (ORDER BY score DESC, player)

player	score	rn	rnk	drnk	pct_beaten
A	90	1	1	1	100.00
B	85	2	2	2	60.00
C	85	3	2	2	60.00
D	70	4	4	3	20.00
E	70	5	4	3	20.00
F	60	6	6	4	0.00

Explanation

- B and C tie at 85: ROW_NUMBER gives 2 and 3, RANK gives 2 and 2 (next is 4), DENSE_RANK gives 2 and 2 (next is 3).
- PERCENT_RANK = (rank - 1) / (n - 1) on ascending score: F 0%, D/E 20%, B/C 60%, A 100%.
- Interview rule of thumb: unique ranks -> ROW_NUMBER; competition ranking with gaps -> RANK; no gaps -> DENSE_RANK.

BigQuery

```
select player, score,
       row_number() over (order by score desc, player) as rn,
       rank()       over (order by score desc)         as rnk,
       dense_rank() over (order by score desc)         as drnk,
       round(100 * percent_rank() over (order by score), 2) as pct_beaten
from scores
order by score desc, player;
```

MySQL 8.0+ equivalent

```
select player, score,
       row_number() over (order by score desc, player) as rn,
       rank()       over (order by score desc)         as rnk,
       dense_rank() over (order by score desc)         as drnk,
       round(100 * percent_rank() over (order by score), 2) as pct_beaten
from scores
order by score desc, player;
```

B25 Churn candidates as of a date [Medium]

As of 2026-10-31, a customer is a churn candidate if they have placed at least 3 orders in total and their last order was MORE than 90 days ago. Return customer_id, orders, last_order_date, days_since_last.

Schema

`orders` (customer_id string, order_date date, amount int)

Sample input

`orders`

customer_id	order_date	amount
c1	2026-02-01	50
c1	2026-03-05	60
c1	2026-05-10	70
c2	2026-07-01	20
c2	2026-08-01	25
c2	2026-09-01	30
c2	2026-10-01	35
c3	2026-01-01	40
c3	2026-01-20	45
c4	2026-06-01	10
c4	2026-07-15	15
c4	2026-08-01	20
c5	2026-04-01	5
c5	2026-05-01	5
c5	2026-06-01	5
c5	2026-07-01	5
c5	2026-08-02	5

Expected output (ORDER BY days_since_last DESC)

customer_id	orders	last_order_date	days_since_last
c1	3	2026-05-10	174
c4	3	2026-08-01	91

Explanation

- c1: 3 orders, last on May 10 -> 174 days -> candidate. c4: last order Aug 1 = exactly 91 days -> candidate.
- c5: last order Aug 2 = exactly 90 days, not MORE than 90 -> excluded. c3 has only 2 orders. c2 ordered on Oct 1 (active).
- Pass the as-of date explicitly instead of CURRENT_DATE so results are reproducible; check the boundary (> vs >=) with a test row.

BigQuery

```
select customer_id, orders, last_order_date, days_since_last
from (
  select customer_id,
         count(*) as orders,
         max(order_date) as last_order_date,
         date_diff(date '2026-10-31', max(order_date), day) as days_since_last
  from orders
  group by customer_id
)
where orders >= 3 and days_since_last > 90
order by days_since_last desc;
```

MySQL 8.0+ equivalent

```sql
select customer_id, orders, last_order_date, days_since_last
from (
  select customer_id,
         count(*) as orders,
         max(order_date) as last_order_date,
         datediff('2026-10-31', max(order_date)) as days_since_last
  from orders
  group by customer_id
) t
where orders >= 3 and days_since_last > 90
order by days_since_last desc;
```

BigQuery vs MySQL translation sheet

Task	BigQuery	MySQL 8.0+
Truncate to month / week	DATE_TRUNC(d, MONTH) / DATE_TRUNC(d, WEEK(MONDAY))	DATE_FORMAT(d,'%Y-%m-01') / DATE_SUB(d, INTERVAL WEEKDAY(d) DAY)
Date difference	DATE_DIFF(later, earlier, DAY)	DATEDIFF(later, earlier)
Timestamp difference	TIMESTAMP_DIFF(later, earlier, MINUTE)	TIMESTAMPDIFF(MINUTE, earlier, later) (order differs!)
Format date	FORMAT_DATE('%Y-%m', d)	DATE_FORMAT(d, '%Y-%m')
Extract part	EXTRACT(HOUR FROM ts), EXTRACT(YEAR FROM d)	HOUR(ts), YEAR(d)
Conditional count	COUNTIF(cond)	SUM(cond) or COUNT(CASE WHEN cond THEN 1 END)
Inline if	IF(c, a, b), IFNULL(x, 0)	IF(c, a, b), IFNULL(x, 0)
Safe divide	SAFE_DIVIDE(a, b) -> NULL if b = 0	a / NULLIF(b, 0)
Filter on window fn	QUALIFY rnk <= 3 (needs WHERE/GROUP BY/HAVING, use WHERE TRUE)	Subquery or CTE, then WHERE rnk <= 3
First / last in group	ARRAY_AGG(x ORDER BY t LIMIT 1)[OFFSET(0)]	FIRST_VALUE / ROW_NUMBER windows
Array to rows	FROM t, UNNEST(arr) AS x	JSON_TABLE(col, '$[*]' COLUMNS (x ... PATH '$'))
Join list into string	STRING_AGG(x, ',' ORDER BY x)	GROUP_CONCAT(x ORDER BY x SEPARATOR ',')
Date series	UNNEST(GENERATE_DATE_ARRAY(a, b))	WITH RECURSIVE cal AS (...)
Pivot	PIVOT(SUM(v) FOR col IN ('A','B'))	SUM(CASE WHEN col='A' THEN v END)
Percentile	PERCENTILE_DISC(x, 0.9) OVER (...) / PERCENTILE_CONT	ROW_NUMBER + CEIL(cnt * p)
String concat	CONCAT(a,b) or a || b	CONCAT(a,b) (|| is OR)
Table names	`project.dataset.table`	db.table
Top rows	LIMIT n	LIMIT n
Integer division	DIV(a, b); a / b is FLOAT64	a DIV b; a / b is decimal
Types	STRING, INT64, FLOAT64, BOOL, TIMESTAMP	VARCHAR, INT, DOUBLE/DECIMAL, TINYINT(1), DATETIME